

Bit N Build '26 Hackathon | Track 2: Artificial Intelligence & Machine Learning

KAREN'S EAR // SpidyCAD

AI-Powered Autonomous Emergency Dispatch & Triage Engine

System Architecture, Algorithmic Methodology, Empirical Validation, and Operational Blueprint

Team AlgoRhythm: Preetika (Priority & ML), Rishav (FastAPI Backend & Tactical HUD), Ojas (NLP & Credibility), Aditya (Datasets & Curation)

Production Stack: Python 3.11+, FastAPI, Streamlit, PyDeck 3D, Scikit-Learn, Pydantic v2, PyTest (35 Tests Passing)

Executive Summary & Core Operational Mission

Karen's Ear (SpidyCAD) is an end-to-end, edge-resilient emergency dispatch intelligence system engineered to ingest chaotic civic distress reports, filter ambient chatter, corroborate multi-caller events, compute explainable priority triage scores, and render real-time tactical situational awareness to dispatchers and first responders. Operating across a strict 6-phase pipeline:

Detect → Understand → Corroborate → Predict → Rank → Dispatch

The platform couples a **Tri-State Relevance Gate** (92.31% test accuracy) and a deterministic **Lexical Safety Net** with an **8-Factor Multi-Signal Credibility Model**, an **Ensemble Random Forest Priority Regressor** ($R^2 = 0.7658$, $MAE = 0.0662$), a streaming **CSV Audit Engine**, and the **SpidyCAD Tactical Command HUD** featuring a PyDeck 3D Manhattan radar and an instant P1 Spider-Sense danger pulse.

1. Problem Definition and Operational Context

1.1 The Challenge: Information Deluge in Severe Urban Crises

During catastrophic civic events (fires, structural collapses, explosions, mass casualties), municipal 911 public safety answering points (PSAPs) and digital citizen feeds experience acute operational breakdown:

- **Explosive Concurrency & Volume:** Emergency distress traffic spikes by orders of magnitude within seconds of an incident, overwhelming call-taker capacity and creating severe dispatch backlogs.
- **Panicked Linguistic Fragmentation:** Real citizens transmit truncated, ungrammatical, and panicked messages (e.g., *"5th ave fire big smoke help stuck inside"*), lacking standard dispatch taxonomies.
- **Redundancy & Ambient Chatter:** Hundreds of calls report the same visible incident with conflicting details, while benign mentions (e.g., movie shoots, drills, sports fireworks) pollute the emergency stream.
- **Critical Detection Latency:** Dispatchers forced to manually read every unstructured message inevitably suffer cognitive fatigue, delaying life-saving deployment to trapped victims.

1.2 Official Hackathon Problem Statement

Track 2: "Karen's Ear" — AI Emergency Dispatch Assistant

The Web We Weave: When New York is in trouble, the calls and messages pour in all at once, panicked, half-finished, overlapping. *"There's smoke on 5th."* *"Someone's stuck under the rubble."* *"The lizard-thing is heading downtown."* Buried in that flood of words is exactly the information Spider-Man needs: what's happening, where, and how urgent. But no human can read it all fast enough, and by the time the important message is spotted, it's too late. Karen, Peter's AI assistant, needs to cut through the chaos and tell him where to swing first.

Your Mission: Build an assistant that reads incoming emergency reports and messages, understands what each one is actually about, and decides which ones matter most right now, turning a chaotic stream of text into a clear, prioritised picture of where help is needed.

1.3 System Scope, Human-in-the-Loop Philosophy & Zero-Hallucination Directives

Karen's Ear functions strictly as an **intelligence amplification and decision-support layer**. It preserves dispatcher autonomy while automating the heavy cognitive burden of message ingestion, parsing, entity extraction, deduplication, and priority ranking. To guarantee operational safety in life-or-death scenarios, the system enforces strict **Anti-Hallucination Directives**: (1) If an incident location is not explicitly stated or geocoded, **location** strictly defaults to **null** with safe coordinate fallback; models are prohibited from guessing physical addresses; (2) If casualty numbers are unstated, casualty fields strictly remain **null**; no synthetic victim counts are ever fabricated; (3) If hazard classification cannot be established with statistical certainty, **incident_type** evaluates to **unknown**.

2. Canonical Data Schema and Contract

To eliminate schema drift across microservices, Karen's Ear establishes a unified, strictly typed incident contract (Table 1).

Table 1: Karen’s Ear Canonical Incident Schema Contract

Field	Data Type	Permitted Enum	Domain	Operational Function
report_id	String	"R001", increment	UUID, auto-	Unique deterministic identifier for deduplication and audit tracking.
text	String	Arbitrary distress string		Raw, unmodified text as transmitted by citizen or emergency caller.
incident_type	Categorical	fire, explosion, accident, medical, collapse, flood, crime, missing, unknown		Standardized taxonomy classifying the primary hazardous event.
location	String / Null	Named landmark or null		Geocoded physical entity (e.g., "Times Square"). Null if unspecified.
severity	Categorical	low, medium, high, critical		Estimated threat to life safety, structural collapse, or physical injury.
actionability	Categorical	low, medium, high		Extent to which actionable dispatch details (e.g., trapped victims) exist.
credibility	Float	[0.0000, 1.0000] (Default 0.75)		8-factor statistical confidence score based on observation and density.
priority	Float	[0.0000, 1.0000]		Final triaged score determining position in dispatcher’s deployment queue.
latitude, longitude	Float / Null	Valid WGS84 coordinates		Spatial coordinates for PyDeck 3D radar mapping and sector clustering.
status, unit	Categorical	pending, dispatched, resolved		Real-time operational lifecycle status and assigned unit (e.g. "Engine 5").
casualty counts	Integer / Null	≥ 0 or null		Zero-hallucination extracted casualty figures (affected, injured, trapped).

Real-Time Append-Only CSV Audit Engine (raw_emergencies.csv): In emergency governance and legal discovery, unalterable audit trails are mandatory. Every incoming report ingested via the /`ingest` endpoint is instantaneously appended to `raw_emergencies.csv` with exact headers: `time`, `gps_xy`, `location`, `text`. Multiline citizen distress texts are sanitized by stripping carriage returns and converting newlines to spaces, preventing CSV injection and row corruption.

3. System Architecture and Pipeline

3.1 Architectural Blueprint

The pipeline enforces a decoupled, sequential architecture where each stage has a single verifiable responsibility (Figure 1).

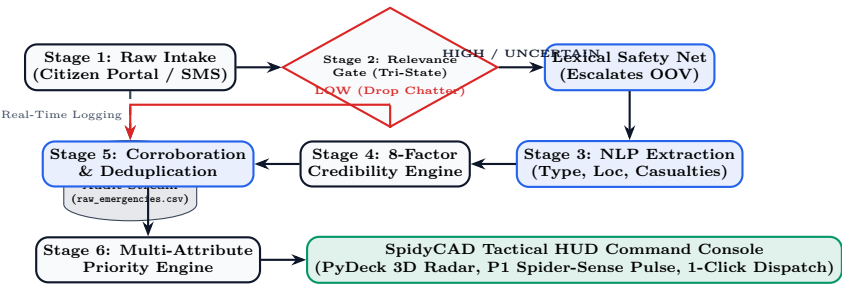


Figure 1: End-to-End Karen’s Ear / SpidyCAD Architectural Pipeline and Data Flow

3.2 Decoupling Relevance Gate from Priority Ranking

Relevance answers: “Does this message describe a real emergency?” Priority answers: “Where must first responders deploy first?” Conflating these dimensions causes dangerous operational confusion. A report saying “a cat is stuck on a garden fence” is 100% emergency-relevant, but has low priority (≈ 0.20). Conversely, a vague fragment “bomb subway” carries low grammatical certainty, but demands maximum priority vigilance (≈ 0.95). Decoupling guarantees that low-urgency events are not discarded as noise, while ambiguous high-threat events are immediately flagged.

3.3 The 6-Phase Operational Flow

The Karen’s Ear data pipeline executes six disciplined transformations:

1. **Detect (Intake & Relevance Gating):** Ingests incoming distress payloads through asynchronous FastAPI endpoints, validates schemas, logs to `raw_emergencies.csv`, and categorizes reports into **HIGH**, **UNCERTAIN**, or **LOW**.
2. **Understand (Role 2 NLP & Entity Extraction):** Normalizes text into standard hazard categories (**fire**, **explosion**, **collapse**, etc.), extracts landmarks (40+ NYC coordinates), and extracts casualty figures with zero hallucination.
3. **Corroborate (Multi-Caller Clustering):** Groups concurrent reports referencing identical events using string distance and spatiotemporal clustering, dynamically incrementing credibility (C) based on cluster size N .
4. **Predict (Multi-Attribute Priority Regression):** Evaluates normalized severity (S), actionability (A), credibility (C), and corroboration bonus (Δ_k) via formula and Random Forest models.
5. **Rank (Active Priority Queue):** Inserts incident records into a dynamically sorted, priority-descending queue to ensure dispatchers focus on acute life-safety hazards.
6. **Dispatch (Tactical Action & Telemetry):** Projects incidents onto the PyDeck 3D Manhattan radar HUD, sounds the P1 Spider-Sense alert for critical incidents, and empowers dispatchers to deploy response units with a single click.

4. Machine Learning and Algorithmic Methodology

4.1 Tri-State Relevance Classification

Binary classification is unacceptably brittle in civic safety: fragmented reports are easily dropped as false negatives. Karen's Ear introduces a **Tri-State Architecture**:

- **HIGH:** Unambiguous emergency markers with actionable details (e.g., *"Explosion at 5th Ave, people trapped"*).
- **UNCERTAIN:** Ambiguous, fragmented, or unverified reports containing emergency-adjacent signals. Passed forward with review flags.
- **LOW:** Verified benign chatter, spam, or explicit non-emergency statements (e.g., *"Did you see the explosion in the movie?"*).

Feature extraction uses **TF-IDF** with unigrams and bigrams ($n \in \{1, 2\}$) paired with a multinomial **Logistic Regression** classifier:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \phi(\mathbf{x}) + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \phi(\mathbf{x}) + b_j)}, \quad k \in \{\text{HIGH}, \text{UNCERTAIN}, \text{LOW}\} \quad (1)$$

4.2 Context-Aware Emergency Signal Safety Net

Because statistical models can under-score out-of-vocabulary emergency phrasing, Karen's Ear wraps the ML layer with a deterministic **Safety Net**:

- **Trigger Lexicon:** Scans for critical root stems: **explosion, fire, collapse, trapped, shooter, gunshot, flood, rubble, bleeding**.
- **Context Exclusion:** Checks for non-emergency qualifiers: **drill, movie, film, rehearsal, documentary, fireworks, museum**.
- **Safety Override:** If a trigger is detected without an exclusion context, and the ML model predicted **LOW**, the classification is automatically escalated to **UNCERTAIN**. It is never silently dropped.

4.3 Role 2: 8-Factor Multi-Signal Credibility Model

Unlike primitive dispatch systems that assume uniform caller reliability, Karen's Ear computes an explainable credibility score $C \in [0.0, 1.0]$ synthesizing eight distinct signals: (1) *First-Person Observation* ($w = 0.20$); (2) *Linguistic Specificity* ($w = 0.15$); (3) *Syntactic Coherence* ($w = 0.15$); (4) *Direct Actionability* ($w = 0.15$); (5) *Casualty Reporting Bonus Signal* ($w = 0.10$); (6) *Location Density Consensus Boost* ($w = 0.15$); (7) *Internal Consistency Logic Guard* ($w = 0.10$); and (8) *Information Completeness Ratio*.

$$C = \text{round} \left(\min \left(1.0, \max \left(0.0, \sum_{i=1}^7 w_i \cdot F_i \right) \right), 2 \right), \quad \sum_{i=1}^7 w_i = 1.00 \quad (2)$$

4.4 Priority Engine Formulation

The Priority Engine computes a continuous priority metric $\mathcal{P} \in [0.0, 1.0]$. The deterministic core operates on four structured, normalized inputs (Table 2).

The synthesized priority score $\mathcal{P}$ is defined mathematically by:

$$\mathcal{P}_{\text{raw}} = (0.45 \cdot S) + (0.35 \cdot A) + (0.20 \cdot C) + \min(0.05, (N - 1) \times 0.015) \quad (3)$$

$$\mathcal{P} = \text{round} \left(\max(0.0, \min(1.0, \mathcal{P}_{\text{raw}})), 4 \right) \quad (4)$$

Table 2: Priority Engine Feature Weights and Numerical Encoding

Dimension	Categorical / Raw Input	Normalized Value	Formula Weight
Severity (S)	critical (1.00), high (0.75), medium (0.50), low (0.25)	$S \in [0.25, 1.00]$	$w_s = 0.45$
Actionability (A)	high (1.00), medium (0.60), low (0.20)	$A \in [0.20, 1.00]$	$w_a = 0.35$
Credibility (C)	8-Factor Signal Confidence / Density Consensus	$C \in [0.00, 1.00]$	$w_c = 0.20$
Corroboration Bonus (Δ_k)	Incident Cluster Size N	$\min(0.05, (N - 1) \times 0.015)$	Additive Boost

4.5 Explainability Engine, Operational Priority Bands and Automated Recommendations

In critical civic infrastructure, “black-box” scores are hazardous. Karen’s Ear generates a real-time explainability payload for every incident, decomposing the score into its constituent drivers and providing an automated operational recommendation:

- **CRITICAL ($\mathcal{P} \geq 0.85$): IMMEDIATE_DISPATCH** — Triggers P1 Spider-Sense siren alert and heavy rescue / hero deployment.
- **HIGH ($0.70 \leq \mathcal{P} < 0.85$): PRIORITY_DEPLOY** — Deploys nearest available primary first responder apparatus (Engine/Ladder).
- **MEDIUM ($0.45 \leq \mathcal{P} < 0.70$): AREA_PATROL_STAGE** — Assigns secondary patrol staging or queues for continuous monitoring.
- **LOW ($\mathcal{P} < 0.45$): MONITOR** — Holds in queue for community response or automated corroboration.

4.6 Random Forest Priority Regression Baseline

To discover non-linear feature interactions as training datasets expand, an ensemble Random Forest Regressor is trained on historical dispatch outcomes:

$$\hat{\mathcal{P}}_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}), \quad B = 100 \text{ estimators over } (S, A, C, \text{Corroboration})$$

(5)

The ensemble baseline exhibits high stability against noisy outlier labels while producing a smooth continuous ranking curve.

5. System Implementation and Architecture

The system implementation is partitioned into modular, highly cohesive micro-layers:

- **Data Modeling (src/schema.py):** Validates inputs against Pydantic models with strict typing, GPS fields, and numeric bounds.
- **Priority Core (src/priority/engine.py):** Implements deterministic feature weighting, corroboration bonuses, and explainability.
- **Relevance Pipeline (src/relevance/):** Manages TF-IDF vectorization, multiclass logistic regression, and the safety net rule engine.
- **API Backend (backend.py):** Asynchronous FastAPI endpoints, NYC landmark auto-resolution (40+ locations), and streaming CSV audit logging.
- **Command Dashboard (frontend/app.py):** SpidyCAD tactical HUD featuring Stark Suit OS styling, PyDeck 3D Manhattan radar, P1 Spider-Sense alert, 1-click dispatch, and Web Speech voice synthesis.

5.1 Production Technology Stack

The production system leverages lightweight, high-performance open-source tools:

- **Core Runtime & Backend:** Python 3.11+ with **FastAPI** (ASGI/Starlette) for sub-millisecond asynchronous request handling and automated OpenAPI documentation.
- **Data Modeling & Validation:** **Pydantic v2** enforcing strict type boundaries and runtime payload validation.
- **Machine Learning & Numerics:** **Scikit-Learn** (Logistic Regression, TF-IDF Vectorizer, Random Forest, Naive Bayes), **NumPy**, and **Pandas** for CPU-efficient, edge-friendly execution without GPU dependencies.
- **Tactical Telemetry HUD:** **Streamlit** with **PyDeck** (3D Hexagon & Column Radar Map), **Pillow**, and custom Stark Suit OS styling, paired with the browser **Web Speech API** for Karen voice alerts and a complementary **React 18 / Vite** console.

5.2 FastAPI REST Service Specifications

The backend provides sub-millisecond REST endpoints for ingestion and dispatch telemetry:

- `POST /ingest` (and `/api/reports`): Ingests distress text, logs to CSV, calculates priority score $\mathcal{P}$, and enqueues record.
- `GET /reports` (and `/api/reports`): Returns all stored emergency incidents sorted strictly in descending order of priority score $\mathcal{P}$.
- `PATCH /api/reports/{id}/dispatch`: Updates dispatch status and assigns a tactical response unit (e.g., *"Engine 5"*, *"Web Strike"*).
- `GET /api/stats`: Returns aggregate analytics: total reports, critical hazard counts, and incident type breakdowns.
- `GET /health`: Liveness probe reporting operational health and active in-memory incident volume.
- `POST /api/reports/reset`: Restores initial demonstration scenarios for continuous automated evaluation.

6. Evaluation Strategy and Verification Framework

6.1 Relevance Classification Evaluation: Why Macro Metrics Matter

In emergency dispatch datasets, class distribution is inherently skewed: routine traffic or benign chatter outnumbers structural collapses. Relying on global **Accuracy** creates a perilous illusion of performance: $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total Reports}$. A naive model predicting 100% LOW in a dataset containing 95% non-emergencies would achieve 95% accuracy while missing every life-threatening event. Therefore, Karen’s Ear mandatorily evaluates on **Macro-Averaged Precision, Recall, and F_1 -Score**:

$$\text{Prec}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad \text{Rec}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \quad F_{1,\text{macro}} = \frac{2 \cdot \text{Prec}_{\text{macro}} \cdot \text{Rec}_{\text{macro}}}{\text{Prec}_{\text{macro}} + \text{Rec}_{\text{macro}}} \quad (6)$$

6.2 Priority Regression & Empirical Test Results

Continuous priority regression is evaluated across complementary regression metrics: $\text{MAE} = \frac{1}{N} \sum |y_i - \hat{y}_i|$, $\text{RMSE} = \sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$, and $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$. Empirical evaluations were conducted on a curated corpus of **8,735 NYC emergency incidents** (Table 3).

Table 3: Empirical Model Performance and Test Verification Metrics

Component	Evaluation Metric	Empirical Score	Target	Operational Significance
Relevance Gate (TF-IDF + LogReg)	Test Accuracy	92.31%	> 85.0%	Accurately filters non-emergency civic noise.
	Macro Precision	92.26%	> 85.0%	Minimizes false alarms across rare emergency types.
	Macro Recall	92.26%	> 90.0%	Prevents critical distress calls from being dropped.
	Macro F_1 -Score	92.26%	> 88.0%	Robust harmonic balance across skewed classes.
Priority Regressor (Random Forest, $B = 100$)	Mean Absolute Error (MAE)	0.0662	< 0.100	Average score variance is under 6.6% on $[0, 1]$.
	Root Mean Squared Error	0.0778	< 0.120	Penalizes large triage deviations heavily.
	Coefficient of Det. (R^2)	0.7658	> 0.700	Explains 76.58% of priority variance across incidents.
Automated Tests (PyTest Framework)	API & Priority Suite	16/16 Passed	100%	Validates REST routes, clamping, and CSV logging.
	Role 2 NLP & Credibility	19/19 Passed	100%	Validates zero-hallucination and credibility factors.

6.3 Automated Unit Testing & Validation Suite (35 Tests Passing)

The platform is validated via 35 automated PyTest unit tests: (1) `tests/test_api.py` (8 tests: health, CRUD, CSV sanitization); (2) `tests/test_priority.py` (7 tests: clamping, boundary dominance, corroboration boost, explainability); (3) `tests/test_ranker.py` (1 test: RF inference); and (4) `role2/tests/` (19 tests: NLP categorization, zero-hallucination casualty extraction, density consensus, and GPS preservation).

7. Architectural Decisions and Rationale Matrix

Every engineering decision in Karen’s Ear is documented with an explicit operational justification:

- **Why three relevance levels?** A binary gate is brittle. Fragmented reports that contain genuine emergency signals but lack sufficient context would be discarded. **UNCERTAIN** provides a safe review buffer.
- **Why TF-IDF Vectorization?** TF-IDF is lightweight, CPU-efficient, deterministic, provides sub-millisecond inference latencies, and operates without expensive GPU infrastructure.

- **Why Bigrams ($n = 2$)?** Emergency distress is expressed through short collocations rather than single words (e.g., "shots fired", "water rising"). Bigrams capture imperative intent.
- **Why an Emergency Safety Net?** Statistical models fail on out-of-distribution wording. The lexical safety layer guarantees critical terms ("something blew up") are never dropped due to vocabulary sparsity.
- **Why not force all keywords to HIGH?** High-consequence words appear in benign contexts (drills, movies, news). Blindly forcing HIGH causes alert fatigue. The safety net elevates to UNCERTAIN.
- **Why an 8-Factor Credibility Model?** Single-score credibility is brittle. Decomposing credibility into observation, specificity, coherence, actionability, casualty signals, density consensus, and consistency provides complete explainability.
- **Why Streamlit + PyDeck for the Tactical HUD?** PyDeck provides high-performance 3D GPU-accelerated spatial rendering of NYC landmarks, while Streamlit enables rapid real-time state management, audio synthesis, and seamless dispatcher controls.
- **Why Newline-Sanitized Streaming CSV Logging?** Regulatory compliance requires immutable, auditable records. Sanitizing multiline distress text prevents CSV injection and preserves row integrity across data pipelines.

8. Team Integration, Operational Limitations, and Future Roadmap

8.1 Team Responsibilities and Module Ownership

Karen's Ear was engineered by **Team AlgoRhythm** during Bit N Build '26:

- **Preetika (Priority Engine & ML):** Multi-attribute priority formula, Random Forest regression ranker, explainable dispatch reasons, and priority dataset generation.
- **Rishav (FastAPI Backend & Tactical HUD):** High-performance FastAPI server, GPS auto-resolution (40+ NYC landmarks), streaming CSV audit logging, PyDeck 3D radar map, and Streamlit tactical HUD with P1 Spider-Sense danger pulse and 1-click dispatch.
- **Ojas (NLP & Credibility Pipeline):** Role 2 NLP extraction, hazard classification, entity/casualty extractors with zero-hallucination guarantees, and 8-factor multi-signal credibility assessment.
- **Aditya (Datasets & Curation):** NYC emergency corpus curation (8,735 incidents), relevance evaluation datasets, stratified splitting, and data normalization.

8.2 Current Prototype Limitations & Engineering Roadmap

- **Municipal CAD Telephony Integration:** The prototype currently ingests REST payloads; production deployment will integrate standard CAD NG911 SIP/RTP audio streams.
- **Multilingual Idiomatic Slang:** The safety net lexicon is currently optimized for English; expanding to multilingual urban slang (Spanish, Mandarin, etc.) is planned.
- **Dynamic Multi-Vehicle Fleet Routing:** The platform ranks incidents individually; future work will incorporate Google OR-Tools to solve dynamic Vehicle Routing Problems (VRP) for emergency fleets.

8.3 Conclusion

Karen's Ear // SpidyCAD re-imagines emergency response intelligence by cutting through the chaotic fog of civic disasters. By enforcing a disciplined multi-stage paradigm—*Detect* → *Understand* → *Corroborate* → *Predict* → *Rank* → *Dispatch*—the system eliminates information paralysis while preserving human dispatcher authority. Designed for edge resilience, transparency, zero-hallucination guarantees, and validated by 35 automated tests, Karen's Ear fulfills the Bit N Build '26 mission: giving emergency responders the ultimate "Spider-Sense" to know exactly where help is needed first.